Name : Adhithya.S
Reg. No: 192412352

CODE:

```
import csv
import os

# Get the folder where this Python file is located
folder = os.path.dirname(os.path.abspath(__file__))

# CSV file path
csv_file = os.path.join(folder, "trainingdata.csv")

# Read training data from CSV file
with open(csv_file, "r", newline="") as file:
    data = list(csv.reader(file))

# Remove empty rows
data = [row for row in data if row]

# Check whether CSV contains data
if len(data) < 2:
    print("Error: CSV file is empty or does not contain training examples.")
    exit()

# Display CSV file being used
print("Reading CSV file from:", csv_file)

# Separate attribute names and training examples
attributes = data[0][:-1]
examples = data[1:]

# Number of attributes
n = len(attributes)

# Find all possible values for each attribute
attribute_values = []

for i in range(n):
    values = set()
    for row in examples:
        values.add(row[i])
    attribute_values.append(values)

# Initialize Specific Boundary S
S = [['Ø'] * n]

# Initialize General Boundary G
G = [['?'] * n]


# Function to check whether hypothesis covers an example
def covers(h, x):

    for i in range(n):

        if h[i] == 'Ø':
            return False

        if h[i] == '?':
            continue

        if h[i] != x[i]:
            return False

    return True
```

```
# Function to check whether h1 is more general than or equal to h2
def more_general_or_equal(h1, h2):

    for i in range(n):

        if h1[i] == '?':
            continue

        elif h1[i] == h2[i]:
            continue

        else:
            return False

    return True


# Remove duplicate hypotheses
def remove_duplicates(hypotheses):

    unique = []

    for h in hypotheses:

        if h not in unique:
            unique.append(h)

    return unique


# Keep only the most specific hypotheses in S
def remove_more_general_from_S(S):

    result = []

    for s in S:

        is_more_general = False

        for other in S:

            if s != other and more_general_or_equal(s, other):
                is_more_general = True
                break

        if not is_more_general:
            result.append(s)

    return remove_duplicates(result)


# Keep only the most general hypotheses in G
def remove_more_specific_from_G(G):

    result = []

    for g in G:

        is_more_specific = False

        for other in G:

            if g != other and more_general_or_equal(other, g):
                is_more_specific = True
                break

        if not is_more_specific:
            result.append(g)

    return remove_duplicates(result)
```

```
# Process each training example
for row in examples:

    # Skip incomplete rows
    if len(row) != n + 1:
        continue

    x = row[:-1]
    label = row[-1].strip().lower()

    print("\nExample:", x, "=>", row[-1])

    # ---------- POSITIVE EXAMPLE ----------
    if label == "yes":

        # Remove hypotheses from G that do not cover the positive example
        G = [g for g in G if covers(g, x)]

        # Generalize S to include the positive example
        new_S = []

        for s in S:

            new_h = s.copy()

            # If hypothesis does not cover x, generalize it
            if not covers(s, x):

                for i in range(n):

                    if new_h[i] == 'Ø':
                        new_h[i] = x[i]

                    elif new_h[i] != x[i]:
                        new_h[i] = '?'

            new_S.append(new_h)

        S = remove_duplicates(new_S)

        # Keep only hypotheses in S that are more specific than G
        S = [
            s for s in S
            if any(more_general_or_equal(g, s) for g in G)
        ]

        S = remove_more_general_from_S(S)

    # ---------- NEGATIVE EXAMPLE ----------
    elif label == "no":

        # Remove hypotheses from S that cover the negative example
        S = [s for s in S if not covers(s, x)]

        new_G = []

        for g in G:

            # If g does not cover the negative example, keep it
            if not covers(g, x):

                new_G.append(g)

            else:

                # Minimal specialization of g
                for i in range(n):

                    if g[i] == '?':
```

```
                for value in attribute_values[i]:

                    # Specialize using a value different from
                    # the negative example
                    if value != x[i]:

                        new_h = g.copy()
                        new_h[i] = value

                        # New hypothesis must remain more general
                        # than at least one hypothesis in S
                        if any(
                            more_general_or_equal(new_h, s)
                            for s in S
                        ):
                            new_G.append(new_h)

        G = remove_duplicates(new_G)

        # Remove overly specific hypotheses
        G = remove_more_specific_from_G(G)

    else:

        print("Warning: Class label must be Yes or No.")
        continue

    # Display boundaries after each example
    print("Specific Boundary S:", S)
    print("General Boundary G:", G)


# ---------------- FINAL OUTPUT ----------------

print("\n" + "=" * 50)
print("FINAL VERSION SPACE BOUNDARIES")
print("=" * 50)

print("\nSpecific Boundary (S):")

for s in S:
    print(s)

print("\nGeneral Boundary (G):")

for g in G:
    print(g)
```

OUTPUT :

```
Example: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'] => Yes
Specific Boundary S: [['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']]
General Boundary G: [['?', '?', '?', '?', '?', '?']]

Example: ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'] => Yes
Specific Boundary S: [['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']]
General Boundary G: [['?', '?', '?', '?', '?', '?']]

Example: ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'] => No
Specific Boundary S: [['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']]
General Boundary G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]

Example: ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change'] => Yes
Specific Boundary S: [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
General Boundary G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]

================================================
FINAL VERSION SPACE BOUNDARIES
================================================

Specific Boundary (S):
['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Boundary (G):
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
```